

SmartAutoTrack : l'essentiel des technologies

PHP · JavaScript · MySQL, puis HTML/CSS, outils et services. À lire la veille : une page par technologie, les questions probables et leurs réponses.

1. La pile technique en un coup d'œil

Technologie	Rôle dans SmartAutoTrack
PHP 8.2	Le cerveau côté serveur : vérifie les droits, applique les règles métier, parle à la base, construit les pages.
MySQL	Stocke les données : utilisateurs, véhicules, interventions, réparations, paiements, abonnements, journal.
JavaScript + jQuery 3.7.1	Rend les pages interactives sans les recharger : fenêtres, AJAX, suivi d'un paiement en direct, contrôles de saisie.
HTML / CSS	La structure et l'apparence : formulaires, charte graphique commune, adaptation au téléphone.
Apache (XAMPP)	Le serveur web qui reçoit les requêtes et exécute PHP.
Composer, PHPUnit, Git	Dépendances, tests automatiques (207 tests), historique et sauvegarde du code sur GitHub.
CamPay, OpenRouter, Brevo, Google	Services externes : paiement Mobile Money, assistant IA, emails, connexion avec Google.

Navigateur (HTML/CSS/JS) → HTTP → Apache → PHP → PDO → MySQL
← page HTML ou réponse JSON

Architecture 3 tiers : présentation (navigateur), application (PHP), données (MySQL). Le navigateur ne touche jamais directement la base : tout passe par PHP, qui contrôle les droits.

Pourquoi PHP + MySQL et pas un framework ou React ?

Simple, gratuit, hébergeable à petit prix sur un hébergement mutualisé au Cameroun, et adapté à la taille du projet. Un framework comme Laravel est l'évolution naturelle (organisation MVC), mais il n'était pas nécessaire au départ.

2. PHP : le langage serveur

Ce que c'est : un langage exécuté sur le serveur. Le visiteur ne voit jamais le code PHP, seulement le résultat (du HTML ou du JSON).

Le cycle d'une page

1. `require_once` `'../config/config.php'` : configuration, session, fonctions communes.
2. `requireRole('client')` : contrôle d'accès ; sinon redirection vers la connexion.
3. Traitement du formulaire (`$_POST`) : jeton CSRF, validation, requête en base.
4. Lecture des données, puis affichage HTML avec `h()` sur chaque valeur.

Les 6 notions à savoir expliquer

Notion	Dans le projet
Session	<code>\$_SESSION['user_id']</code> et <code>['role']</code> mémorisent l'utilisateur connecté entre les pages (cookie HttpOnly). À la connexion : <code>session_regenerate_id()</code> contre le vol de session.
PDO + requête préparée	Accès à MySQL. Les valeurs passent par <code>?</code> et <code>execute()</code> , jamais collées dans le SQL : protection contre l' injection SQL .
Transaction	<code>beginTransaction()</code> / <code>commit()</code> / <code>rollback()</code> : la clôture d'une réparation fait plusieurs écritures « tout ou rien ».
Mots de passe	<code>password_hash()</code> à l'enregistrement, <code>password_verify()</code> à la connexion. Jamais stockés en clair.
CSRF	Un jeton secret dans chaque formulaire, vérifié par <code>verifyCSRFToken()</code> : un autre site ne peut pas faire agir l'utilisateur à son insu.
XSS	<code>h()</code> (<code>htmlspecialchars</code>) sur tout affichage : un texte saisi ne peut pas devenir du code exécuté.

```
$stmt = $conn->prepare("SELECT * FROM vehicule WHERE idVehicule = ? AND idClient = ?");
$stmt->execute([$vehiculeId, $_SESSION['user_id']]); // le client ne voit que SES véhicules
```

À retenir

- `includes/` contient la logique métier partagée (paiements, abonnements, échéances, journal, PDF).
- `ajax/` contient des fichiers qui répondent en **JSON** au JavaScript.
- Toute vérification importante est faite **côté serveur**, même si le JavaScript l'a déjà faite.

Différence entre `include` et `require_once` ?

`require` arrête le script si le fichier manque, `include` continue avec un avertissement. Le suffixe `_once` évite de charger deux fois le même fichier. On utilise `require_once` pour la configuration et les fonctions.

Comment empêchez-vous un client de voir le véhicule d'un autre ?

Chaque requête filtre aussi sur l'utilisateur connecté (`AND idClient = ?` avec l'identifiant de session). Changer le numéro dans l'adresse ne suffit donc pas : la requête ne renvoie rien.

Comment PHP appelle-t-il CamPay ou l'IA ?

Avec **cURL**, une requête HTTP envoyée depuis le serveur, en JSON (`includes/http_client.php`). Les clés restent ainsi sur le serveur, jamais dans le navigateur.

3. JavaScript : l'interactivité

Ce que c'est : un langage exécuté dans le **navigateur**. Il modifie la page sans la recharger. Le projet utilise du JavaScript natif et **jQuery 3.7.1** (écriture plus courte : `$()`, `.on()`, `$.ajax()`).

Les 5 notions à savoir expliquer

Notion	Dans le projet
DOM	La page vue comme des objets que le JS lit et modifie (<code>document.querySelector</code> , <code>classList</code>) : ouvrir une fenêtre, afficher un message.
Événements	<code>addEventListener('click', ...)</code> : réagir à un clic, une saisie, l'envoi d'un formulaire.
AJAX	Le JS appelle un fichier <code>ajax/*.php</code> en arrière-plan et reçoit du JSON , sans recharger la page (notifications, messagerie, assistant IA, paiement).
Polling	Pendant un paiement Mobile Money, la page redemande le statut toutes les quelques secondes jusqu'à « Payé » ou « Échoué ».
textContent	Pour afficher un texte venant des données, on utilise <code>textContent</code> et non <code>innerHTML</code> : le texte n'est jamais interprété comme du code (protection XSS).

```
$.ajax({ url: SITE_URL + 'ajax/campay_status.php', method: 'POST',
  data: { paiement_id: id } }) // le jeton CSRF est ajouté automatiquement (main.js)
.done(function (r) { if (r.statut === 'PAYE') { /* afficher le reçu */ } });
```

À retenir

- `assets/js/main.js` : code commun à toutes les pages (jeton CSRF des appels AJAX, règles de mot de passe en direct, filtres de saisie, fenêtre du rapport, menu mobile).
- Le JS améliore le confort ; il **ne remplace jamais** la vérification serveur, car l'utilisateur peut le désactiver ou le modifier.

Pourquoi jQuery ?

Il simplifie les sélections, les événements et les appels AJAX, et fonctionne de la même façon dans tous les navigateurs. Le reste est en JavaScript natif.

Qu'est-ce que le JSON ?

Un format texte d'échange de données, par exemple `{"success": true, "statut": "PAYE"}`. PHP le produit avec `json_encode()`, le JavaScript le lit directement.

Le contrôle du mot de passe en JavaScript suffit-il ?

Non. Il aide l'utilisateur à corriger en direct, mais les mêmes règles sont revérifiées en PHP (`includes/password_policy.php`).

4. MySQL : la base de données

Ce que c'est : un système de gestion de base de données **relationnelle** : des tables reliées entre elles. Base : `charles`, moteur **InnoDB** (clés étrangères et transactions), encodage **utf8mb4** (accents et emojis).

Les tables principales

`utilisateur` (comptes) → `client`, `garage`, `technicien`, `administrateur` (un rôle par table); `vehicule`; `intervention`; `anomalie`; `reparation`; `paiement`; `abonnement`; `notifications`; `journalactivites`; `entretien`.

Les 6 notions à savoir expliquer

Notion	Explication courte
Clé primaire	Identifiant unique d'une ligne (<code>idVehicule</code>), souvent en <code>AUTO_INCREMENT</code> .
Clé étrangère	Lien vers une autre table (<code>vehicule.idClient</code> → <code>client</code>): MySQL refuse un lien vers une ligne qui n'existe pas.
ON DELETE	<code>CASCADE</code> supprime aussi les lignes liées; <code>SET NULL</code> les garde en vidant le lien; <code>RESTRICT</code> interdit la suppression.
Types	<code>DECIMAL</code> pour l'argent (pas d'erreur d'arrondi), <code>DATETIME</code> , <code>ENUM</code> pour les statuts (<code>PLANIFIEE</code> , <code>EN_COURS</code> , <code>TERMINEE</code> ...).
Jointure	<code>JOIN</code> réunit des tables liées; <code>LEFT JOIN</code> garde la ligne même s'il n'y a pas de correspondance (ex. <code>intervention</code> sans <code>garage</code>).
Transaction + FOR UPDATE	<code>SELECT ... FOR UPDATE</code> verrouille la ligne : deux clics simultanés ne peuvent pas créer deux paiements ou deux affectations.

```
SELECT i.idIntervention, v.marque, v.immatriculation, g.nomGarage
FROM intervention i
JOIN vehicule v      ON v.idVehicule = i.idVehicule
LEFT JOIN garage g   ON g.idGarage   = i.idGarage
WHERE i.idClient = ? AND i.statut IN ('PLANIFIEE', 'EN_COURS');
```

À retenir

- Les évolutions de structure passent par `scripts/migrate_structure.php` : simulation d'abord, puis `-- apply`, relançable sans danger.
- Sauvegarde : export SQL depuis phpMyAdmin.
- Les anciens fichiers `db_init.sql`, `sql/schema.sql` et `charles (1).sql` décrivent l'**ancien** schéma.

Que se passe-t-il si on supprime un client ?

Ça dépend des clés étrangères : ses véhicules ont une contrainte `RESTRICT`, donc la suppression est refusée tant qu'il a des véhicules; d'autres données liées sont en `CASCADE` (supprimées avec lui) ou `SET NULL`.

Pourquoi une base relationnelle et pas MongoDB ?

Les données sont très liées (`client` → `véhicule` → `intervention` → `réparation` → `paiement`) et demandent des transactions fiables pour l'argent. C'est exactement le point fort d'une base relationnelle.

Qu'est-ce qu'un index ?

Une structure qui accélère les recherches sur une colonne, comme l'index d'un livre. Les clés primaires et étrangères sont indexées, et un index sur la date du journal accélère le comptage des nouveautés.

5. HTML, CSS, outils et services

HTML et CSS

- **HTML** : la structure (formulaires avec `required`, `type="email"`, `placeholder`).
- **CSS** : l'apparence. Variables de couleur dans `:root` (`--a-accent: #3956E8`) : une charte commune aux 4 espaces.
- **Flexbox / Grid** pour la mise en page, **@media** pour le téléphone.
- Polices Google Fonts (Sora, Manrope), icônes Font Awesome.
- `asset_url()` ajoute `?v=date` aux fichiers pour que le navigateur recharge les styles modifiés.

Outils

- **XAMPP** : Apache + MySQL + PHP en local.
- **.htaccess** : interdit l'accès web aux dossiers sensibles (`config/`, `uploads/`, `scripts/`).
- **Composer** : installe les dépendances (PHPUnit) dans `vendor/`, non versionné.
- **PHPUnit** : 207 tests automatiques (prix, échéances, mots de passe, PDF...).
- **Git / GitHub** : historique, commit, push, pull, fusion ; `config/local.php` (les secrets) n'est jamais envoyé.

Services externes (API)

Service	Usage	Point clé
CamPay	Paieement Mobile Money (MTN, Orange) des réparations et abonnements	Le webhook (<code>webhooks/campay.php</code>) confirme le paiement ; sa signature est vérifiée et le statut est redemandé à CamPay.
OpenRouter	Assistant IA	Appel côté serveur ; modèles gratuits avec modèles de secours si le premier est saturé.
Brevo	Emails (mot de passe oublié, rappels)	Envoi par API avec une clé secrète.
Google OAuth 2.0	« Continuer avec Google »	Google confirme l'identité ; l'adresse de retour doit être déclarée chez Google.

Qu'est-ce qu'une API et un webhook ?

Une **API** est une porte d'accès qu'un service offre aux autres programmes : on lui envoie une requête HTTP et il répond en JSON. Un **webhook** est l'inverse : c'est le service (CamPay) qui appelle notre serveur pour le prévenir qu'un paiement est terminé.

Où sont rangées les clés secrètes ?

Dans `config/local.php`, sur le serveur uniquement, jamais dans le code envoyé sur GitHub ni dans le JavaScript.

6. Sécurité : la réponse à « Comment avez-vous sécurisé l'application ? »

»

Menace	Protection	Où
Injection SQL	Requêtes préparées PDO	Toutes les requêtes
XSS (code injecté dans la page)	<code>h()</code> en PHP, <code>textContent</code> en JS	Tous les affichages
CSRF (action forcée depuis un autre site)	Jeton dans chaque formulaire et appel AJAX	<code>verifyCSRFToken()</code>
Vol de mot de passe	<code>password_hash()</code> , règles de robustesse	<code>includes/password_policy.php</code>
Force brute sur la connexion	Limitation des tentatives par email et par adresse IP	<code>includes/login_throttle.php</code>
Accès aux données d'un autre	Contrôle du rôle et de la propriété de chaque ressource	<code>requireRole()</code> , <code>AND idClient =</code> ?
Faux paiement	Montant recalculé côté serveur, webhook signé, statut redemandé à CamPay	<code>includes/payments.php</code>
Fuite des clés	Secrets hors du dépôt Git	<code>config/local.php</code>

Les 8 questions à préparer absolument

1. **Présentez votre architecture.** — 3 tiers : navigateur (HTML/CSS/JS), serveur (Apache + PHP), base (MySQL) ; le navigateur ne parle jamais à la base directement.
2. **Comment le frontend communique avec le backend ?** — Par HTTP : pages chargées classiquement, et appels AJAX qui reçoivent du JSON.
3. **Comment gérez-vous les rôles ?** — Le rôle est déduit en base à la connexion, gardé en session, et vérifié sur chaque page par `requireRole()`.
4. **Comment se passe un paiement ?** — Le client saisit son numéro, CamPay envoie une demande sur son téléphone, il confirme ; le webhook et le suivi de statut passent le paiement à « Payé » ; un reçu PDF est généré.
5. **Pourquoi une transaction ?** — Pour que plusieurs écritures liées (réparation, clôture, anomalies, journal) réussissent ou échouent ensemble.
6. **Que fait l'IA exactement ?** — Elle répond aux questions et rappelle les échéances ; elle reçoit les dates déjà calculées par l'application et n'invente rien.
7. **Comment testez-vous ?** — 207 tests PHPUnit automatiques sur les règles métier, plus des tests manuels des parcours.
8. **Quelles améliorations ?** — Hébergement en HTTPS, application installable (PWA), passage à Laravel (MVC), fonctions Premium restantes (carnet d'entretien, tableau de flotte).

Phrase de conclusion : « SmartAutoTrack repose sur une architecture 3 tiers simple et sécurisée : PHP applique les règles et les droits, MySQL garantit l'intégrité des données, et JavaScript rend l'expérience fluide, tandis que des services spécialisés gèrent le paiement Mobile Money, l'IA et les emails. »